Algorithm for file updates in Python

Project description

Create an algorithm using Python to verify if the allow list contains any IP addresses that are on the remove list, and remove those matching IP addresses from the allow list. This is for a hypothetical scenario that requires me to regularly update a file that identifies the employees in a company who can access restricted content.

Open the file that contains the allow list

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Read contents of the file
with open(import_file, "r") as file:
```

I first stored the file name `"allow_list.txt"` in the import_file variable. Then, I used a with statement with the `.open()` function in read mode `("r")` to open the file. This allows the program to access the IP addresses stored in the allow list. The with statement also automatically closes the file after the code inside it finishes running.

Read the file contents

```
with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()
```

I used the `.read()` method to read the contents of the allow list file. This converts the file contents into a string, which I stored in the `ip_addresses` variable. This allows the program to work with the IP addresses and process them in the following steps.

Convert the string into a list

```
# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()
```

I then used the `.split()` method to convert the `ip_addresses` string into a list. The IP addresses in the file are separated by whitespace, so `.split()` separates them into individual elements. Using a list makes it easier to check for and remove specific IP addresses.

Iterate through the remove list

```
# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

    # Display `element` in every iteration
```

Next, I used a for loop to iterate through each IP address in the `remove_list`. During each iteration, the current IP address is stored in the element variable. This allows the algorithm to check each IP address one at a time.

Remove IP addresses that are on the remove list

```
for element in ip_addresses:

  # Build conditional statement
  # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)
```

Inside the for loop, I checked whether the current element exists in the `ip_addresses` list. This check is important because using `.remove()` on an IP address that does not exist in the list would cause an error. If the IP address is found, `.remove()` removes it from the allow list.

Update the file with the revised list of IP addresses

```
# Convert `ip_addresses` back to a string so that it can be written into the text file
ip_addresses = " ".join(ip_addresses)
# Build `with` statement to rewrite the original file
with open(import_file, "w") as file:
  # Rewrite the file, replacing its contents with `ip_addresses`
  file.write(ip_addresses)
```

After removing the required IP addresses, I used the `.join()` method to convert the updated `ip_addresses` list back into a string. I used `" "` as the separator so that each IP address is separated by a space. Finally, I opened `"allow_list.txt"` in write mode `("w")` and used `.write()` to replace the old file contents with the updated list.

Summary

The algorithm removes IP addresses from the allow list that are also found in `remove_list`. It first opens and reads the file, converts the contents into a list, checks each IP address in the remove list, and removes matching addresses. Finally, it converts the updated list back into a string and writes it back to `"allow_list.txt"`.